RateQ: Formal Model, Correctness, Instance Optimality and Exact Maintenance of Cheapest-Feasible-Service Queries

Technical companion to the RateQ paper (artifact document)

Abstract

This document gives complete proofs for the guarantees claimed for RateQ. We formalise rate rule programs exactly as the reference implementation evaluates them, including minimum billable tiers, dimensional weight, rounding rules, extrapolation, surcharges with eligibility predicates, fee groups, fuel and percentage-of-total rules. We prove that the suffix-minimum hull bound never exceeds the true cost (Lemma 13), also in its surcharge-class refinement (Lemma 15). We prove that the threshold query returns exactly the brute-force answer with the smaller-identifier tie rule, and that it is instance optimal in a precisely stated access model (Theorem 18). We prove that the top-2 repair over a support set yields exactly the state of a full recomputation for cell, band, table, rule, zone and fuel edits (Theorem 32), and we state construction, space, query and maintenance bounds. Results on envelope complexity and on general linear predicates are proved where possible and stated as conjectures otherwise. Appendix A maps every definition to the code, and Appendix B lists every point where the code and the theory differ.

1 Model

1.1 Parcels and features

A parcel is $x = (\kappa, a, L, M, S, res, d)$ with integer key $\kappa = origin \cdot 10^5 + zip$, actual weight $a \geq 0$, sorted dimensions $L \geq M \geq S \geq 0$, residential flag *res* and ship day $d \in \mathbb{Z}$. Its *parcel-level features* are L, M, S, $G = L + 2M + 2S$, $V = LMS$, $L + M + S$, a, *res*, *zip*, d and the month of d. Three further features depend on the service s: the billable weight T, the zone *ZONE* and the delivery-area level *DAS*. A *predicate* is any Boolean function of the feature vector. The implementation uses disjunctions of conjunctions of threshold, set-membership and range conditions. No result below requires monotonicity of predicates.

1.2 Rate rule programs

Services are identified by integers $s \in \mathcal{S} = \{0, \ldots, n-1\}$. We fix the tolerance $\epsilon = 10^{-9}$ that the implementation uses for weight comparisons.

Definition 1 (Rate rule program)**.** A service s consists of the following components.

(i) *Zone map.* Starts $\zeta_{s,1} < \cdots < \zeta_{s,R_s}$ and labels $\ell_{s,i} \in \{0, 1, \ldots\} \cup \{\perp\}$. Then $Z_s(\kappa) = \ell_{s,i}$ for $\kappa \in [\zeta_{s,i}, \zeta_{s,i+1})$, with $\zeta_{s,R_s+1} = +\infty$, and $Z_s(\kappa) = \perp$ for $\kappa < \zeta_{s,1}$. The intervals are closed on the left and open on the right, and $\perp$ means that the key is not served.

(ii) *Delivery-area map* $DAS_s(\kappa)$, defined in the same way, with value 0 below the first start.

(iii) *Group key* $\gamma(s) = (D_s, \theta_s, \rho_s, \phi_s, q_s, carry_s, mode_s)$: divisor, dimensional threshold, rounding rule, fuel class and the rounding grid, carry and weight mode of the enterprise rule language.

(iv) *Pre-tier billable weight* $t_s(x) = \tau_{\gamma(s)}(a, V)$, where

- $\rho =$ carrier: $\tau = \max(a, [V > \theta]\, V/D)$;
- $\rho =$ erp-style: $\tau = \max(\bar{a}, \bar{v})$, where $\bar{a}$ rounds a down to 0.1 and then up to an integer, and $\bar{v}$ is the analogous rounding of $\lceil 1000V/D \rceil / 1000$ if this exceeds 1, the value 1 otherwise, and 0 unless $V > \theta$;
- $\rho =$ erp: $\tau = r_{q,carry}(y)$ with $y = \max(a, v)$, a or v for modes $1, 2, 3$, where $v = V/D$ if $D > 0$ and $v = 0$ otherwise, and $r_{q,carry}$ rounds up or half up to the grid q.

(v) *Minimum billable tiers* $(\pi_s^{(2)}, m_s^{(2)})$ (lower tier) and $(\pi_s^{(1)}, m_s^{(1)})$, each optional. Starting from $t = t_s(x)$, the tiers are applied in this order as $t \leftarrow \max(t, m_s^{(j)})$ if $\pi_s^{(j)}$ holds on the feature vector with $T = t$. The result is the *effective weight* $\hat{t}_s(x)$.

(vi) *Rate table of upper-closed threshold pieces.* Breakpoints $b_{s,1} < \cdots < b_{s,K_s}$ define the bands $(b_{s,k-1}, b_{s,k}]$, with $b_{s,0} = -\infty$, and prices $P_s[z,k] \in \mathbb{N} \cup \{\infty\}$ for every zone z and band k. The band of a weight t is

$$k_s(t) = \min\big(\{k : b_{s,k} \geq t - \epsilon\} \cup \{K_s\}\big).$$

(vii) *Extrapolation.* Either none, or a limit λ_s above which the base price is extrapolated linearly from the top band.

(viii) *Ineligibility predicate* ψ_s, which may be absent.

(ix) *Surcharge rules* $r \in \mathcal{R}_s$, each with a predicate π_r, a kind, a key (g_r, φ_r) consisting of a fee group g_r and a fuel flag $\varphi_r \in \{0, 1\}$, per-zone amounts $A_r[z]$ and a coefficient $coef_r$. The kinds are *fixed* (amount $A_r[z]$), *per weight* (amount $[\![coef_r\, \hat{t}_s]\!]$), *percentage of base* (amount $[\![coef_r B_s/100]\!]$) and *percentage of total* (coefficient $coef_r$). Here $[\![\cdot]\!]$ rounds to the nearest integer, with ties to even.

The fuel schedule of class ϕ consists of starts $\sigma_{\phi,0} < \cdots < \sigma_{\phi,W_\phi - 1}$ and ratios $f_{\phi,w}$ in basis points. The ratio on day d is $f_\phi(d) = f_{\phi,\max(0,w(d))}$, where $w(d) = \max\{w : \sigma_{\phi,w} \leq d\}$ and $w(d) = -1$ if no start precedes d. Thus the first segment extends to $-\infty$. A class without a schedule has $f \equiv 0$.

Definition 2 (Cost)**.** Fix x and s, and let $z = Z_s(\kappa)$, $\hat{t} = \hat{t}_s(x)$ and $f = f_{\phi_s}(d)$. All predicates of rules and ψ_s are evaluated on the feature vector with $T = \hat{t}$, $ZONE = z$ and $DAS = DAS_s(\kappa)$. The *base price* is

$$B_s(x) = \begin{cases} \big\lceil \min(P_s[z, K_s]\, \hat{t}/b_{s,K_s},\, 10^{15}) - 10^{-6} \big\rceil & \text{if } s \text{ extrapolates and } \hat{t} > \lambda_s, \\ P_s[z, k_s(\hat{t})] & \text{otherwise.} \end{cases}$$

In the first case $B_s = \infty$ if $P_s[z, K_s] = \infty$. Service s is *eligible* for x iff $z \neq \bot$, $\psi_s(x)$ is false, $B_s(x) < \infty$, and either s extrapolates or $\hat{t} \leq b_{s,K_s}$. For a key (g, φ) let $\mu_{g,\varphi} = \max\big(\{0\} \cup \{\alpha_r : r \text{ fires},\ (g_r, \varphi_r) = (g, \varphi),\ r \text{ not percentage of total}\}\big)$, where α_r is the amount of r. Let $\Sigma^{\mathsf{F}} = \sum_g \mu_{g,1}$, $\Sigma^N = \sum_g \mu_{g,0}$, and let Q be the sum of the coefficients of the firing percentage-of-total rules. Then

$$c_s^0(x) = \left\lfloor \frac{(B_s + \Sigma^{\mathsf{F}})(10^4 + f) + 5000}{10^4} \right\rfloor + \Sigma^N, \qquad c_s(x) = c_s^0(x) + [\![c_s^0(x)\, Q/100]\!], \tag{1}$$

if s is eligible, and $c_s(x) = \infty$ otherwise.

Definition 3 (CFS query)**.** Order pairs lexicographically: $(c, s) <_{\text{lex}} (c', s')$ iff $c < c'$, or $c = c'$ and $s < s'$. Let $F(x) = \{(c_s(x), s) : c_s(x) < \infty\}$. Then $\text{CFS}(x) = \min_{\text{lex}} F(x)$ if $F(x) \neq \emptyset$, and $\text{CFS}(x) = \perp$ otherwise. The implementation encodes $\perp$ as $(\infty, -1)$.

1.3 Assumptions

Every result states which of the following assumptions it uses. Appendix A reports the check of each assumption against the instances.

Assumption A1 (Nonnegativity)**.** Finite prices and fixed amounts are nonnegative. The coefficients of all surcharge kinds and all fuel ratios are nonnegative. Moreover $a, V \geq 0$.

Assumption A2 (Magnitude)**.** Finite prices are at most 10^{15}, and every finite cost is smaller than the sentinel 2^{60} that encodes ∞.

Assumption A3 (Extrapolation limit)**.** If s extrapolates, then $\lambda_s \geq b_{s,K_s}$.

Assumption A4 (Identifiers)**.** The service list is ordered by identifier, and the service at position i has identifier i.

Assumption A5 (Tier predicates)**.** Tier predicates read only parcel-level features, that is, not T, *ZONE* or *DAS*. No tier minimum $m_s^{(j)}$ lies in $(b, b + \epsilon]$ for a breakpoint b of s.

Assumption A6 (Breakpoint spacing)**.** Distinct elements of E_γ (Definition 7) differ by more than ϵ.

Assumption A7 (Shared delivery areas)**.** If the surcharge-class index treats *DAS* as a parcel-level feature for a group, then all services of the group have the same delivery-area map.

Assumptions A1–A4 are used for the query results. Assumptions A5–A6 are used only for maintenance, and Assumption A7 only for the surcharge-class index.

2 The Index

Definition 4 (Key partition and signatures)**.** Let $C = \{0\} \cup \bigcup_s \{\zeta_{s,i}\}$ with elements $u_1 < \cdots < u_N$. The elementary key intervals are $[u_i, u_{i+1})$. The *signature* of interval i is the vector $\sigma_i = (Z_s(u_i))_{s \in \mathcal{S}}$, and Σ is the set of distinct signatures. For a group γ, the *local signature* is the restriction $\sigma|_\gamma$, and Σ_γ is the set of distinct local signatures.

Proposition 5 (Signature partition, P1)**.** *Every Z_s is constant on every elementary key interval, and $|\Sigma| \leq N \leq 1 + \sum_s R_s$. Point location takes $O(\log N)$ time, and the bound on N is attained when all starts are distinct and positive.*

Proof. Every start of every zone map is a cut, so no zone map changes inside an elementary interval. Keys below $u_1 = 0$ do not occur. The cuts number at most $1 + \sum_s R_s$, and Σ is a set of interval labels. Binary search over the cuts locates an interval. If all starts are distinct and positive, all $1 + \sum_s R_s$ cuts are distinct. □

Lemma 6 (Group invariance, L2)**.** *For all services s, s' in the same group, $t_s(x) = t_{s'}(x)$ and $f_{\phi_s}(d) = f_{\phi_{s'}}(d)$ for every parcel. Hence the order of any list of services of one group by (h, s) (Definition 9) does not depend on the fuel schedule, and a fuel edit changes no list of the index.*

Proof. The function τ_γ of Definition 1(iv) reads only the parameters in γ, and the fuel ratio reads only $\phi \in \gamma$. The lists store h, which does not involve f. □

Definition 7 (Elementary weight intervals)**.** For a group γ, let $E_\gamma = \bigcup_{s\in\gamma}\{b_{s,1},\dots,b_{s,K_s}\}$ with elements $E_1 < \cdots < E_m$, and let $E_0 = -\infty$ and $E_{m+1} = +\infty$. The elementary interval of a weight t is $e_\gamma(t) = \min\{e \in \{1,\dots,m+1\} : E_e \geq t - \epsilon\}$.

Lemma 8 (Band–interval correspondence)**.** *Let $s \in \gamma$, let t be a weight and let $e = e_\gamma(t)$. Then:*

(a) $\{k : b_{s,k} \geq t - \epsilon\} = \{k : b_{s,k} \geq E_e\}$. *In particular, if this set is nonempty, then* $k_s(t) = \min\{k : b_{s,k} \geq E_e\}$ *and* $b_{s,k_s(t)-1} < E_e \leq b_{s,k_s(t)}$.

(b) $j_s(e) := \min\{k : b_{s,k} \geq E_e - \epsilon\}$ *satisfies* $j_s(e) \leq k_s(t)$ *whenever the set in (a) is nonempty.*

(c) e_γ *and* k_s *are nondecreasing, and* $e_\gamma(\max(t,t')) = \max(e_\gamma(t), e_\gamma(t'))$.

(d) If the set in (a) is empty, then $t - \epsilon > b_{s,K_s}$.

Proof. (a) If $b_{s,k} \geq t-\epsilon$, then $b_{s,k}$ is an element of E_γ that is at least $t-\epsilon$; since E_e is the least such element, $b_{s,k} \geq E_e$. Conversely, $b_{s,k} \geq E_e \geq t - \epsilon$. For $e = m+1$ both sets are empty. The second claim follows because $k_s(t)$ is the least element of the set. (b) $b_{s,k} \geq E_e$ implies $b_{s,k} \geq E_e - \epsilon$. (c) Both functions are minima of sets that shrink as t grows, and the identity follows from monotonicity. (d) If no breakpoint is at least $t - \epsilon$, then in particular $b_{s,K_s} < t - \epsilon$. □

Definition 9 (Hull bound and lists)**.** For $s \in \gamma$, a zone z and $e \in \{1,\dots,m+1\}$, let

$$h_s(z,e) = \begin{cases} \infty & z = \bot, \\ \min_{k \geq j_s(e)} P_s[z,k] & e \leq m \text{ and } j_s(e) \text{ is defined}, \\ P_s[z,K_s] & \text{otherwise, if } s \text{ extrapolates}, \\ \infty & \text{otherwise.} \end{cases}$$

For $\sigma \in \Sigma_\gamma$ and e, the list $\mathcal{L}_\gamma(\sigma,e)$ contains all services of γ sorted by $(h_s(\sigma_s,e), s)$. For a parcel x, let $z_s = Z_s(\kappa)$, $e_x = e_\gamma(t_\gamma(x))$ and $h_s(x) = h_s(z_s, e_x)$, and define the fuel-scaled bound

$$\beta_s(x) = \left\lfloor \frac{h_s(x)\,(10^4 + f) + 5000}{10^4} \right\rfloor, \qquad \beta_s(x) = \infty \text{ if } h_s(x) = \infty.$$

Remark 10. For an extrapolating service and $e \leq m$ with $j_s(e)$ defined, the implementation also takes the minimum with $P_s[z,K_s]$. This has no effect, because $K_s \geq j_s(e)$.

3 Lower Bound Correctness (L3)

Lemma 11 (Base bound)**.** *Assume A1–A3. Let $s \in \gamma$ be eligible for x, and let t'' be any weight with $t'' \leq \hat{t}_s(x)$. Then $B_s(x) \geq h_s(z_s, e_\gamma(t''))$.*

Proof. Write $\hat{t} = \hat{t}_s(x)$, $e = e_\gamma(t'')$, $z = z_s \neq \bot$, $P = P_s[z,\cdot]$ and $K = K_s$. We first note a fact about extrapolation. If s extrapolates and $\hat{t} > \lambda_s$, then $\hat{t} > b_{s,K}$ by A3, so $P[K]\hat{t}/b_{s,K} \geq P[K]$. Since $P[K] \leq 10^{15}$ by A2, the minimum with 10^{15} is at least $P[K]$. Since $P[K]$ is an integer, $\lceil y - 10^{-6} \rceil \geq P[K]$ for every $y \geq P[K]$. Hence

$$B_s(x) \geq P[K] \qquad \text{whenever } s \text{ extrapolates and } \hat{t} > \lambda_s. \tag{2}$$

Case 1: $e \le m$ and $j = j_s(e)$ is defined. Then $h = \min_{k \ge j} P[k]$. If s extrapolates and $\hat{t} > \lambda_s$, then (2) and $K \ge j$ give $B_s \ge P[K] \ge h$. Otherwise $B_s = P[k_s(\hat{t})]$. If $\{k : b_{s,k} \ge t'' - \epsilon\}$ is nonempty, then $j \le k_s(t'') \le k_s(\hat{t})$ by Lemma 8(b),(c). If it is empty, then $k_s(\hat{t}) = K \ge j$, because $t'' \le \hat{t}$ and the set for $\hat{t}$ is empty as well. In both cases $B_s = P[k_s(\hat{t})] \ge \min_{k \ge j} P[k] = h$.

Case 2: $e = m+1$, or $j_s(e)$ is undefined. We claim $\hat{t} - \epsilon > b_{s,K}$. If $e = m+1$, then $E_m < t'' - \epsilon$ and $b_{s,K} \le E_m$. If $e \le m$ and $j_s(e)$ is undefined, then $b_{s,K} < E_e - \epsilon < E_e$. Since $b_{s,K} \in E_\gamma$, we get $b_{s,K} \le E_{e-1} < t'' - \epsilon$ by the definition of e. In both cases $\hat{t} - \epsilon \ge t'' - \epsilon > b_{s,K}$. Hence $\hat{t} > b_{s,K}$, and a non-extrapolating s would be ineligible. So s extrapolates and $h = P[K]$. If $\hat{t} > \lambda_s$, then (2) applies. Otherwise no breakpoint is at least $\hat{t} - \epsilon$, so $k_s(\hat{t}) = K$ and $B_s = P[K] = h$. □

Lemma 12 (Monotone rounding)**.** *For an integer f, let*

$$\eta_f(y) = \left\lfloor \frac{y(10^4 + f) + 5000}{10^4} \right\rfloor.$$

If $f \ge 0$, then η_f is nondecreasing on the integers and satisfies $\eta_f(y) \ge y$ for every integer $y \ge 0$. Monotonicity also holds for every $f \ge -10^4$.

Proof. For $f \ge -10^4$ the argument of the floor is nondecreasing in y, and the floor is nondecreasing. For $f \ge 0$ and $y \ge 0$ we have $y(10^4 + f) + 5000 \ge 10^4 y$. □

Lemma 13 (Lower bound correctness, L3)**.** *Assume A1–A3. For every parcel x and service s,*

$$h_s(x) \le \beta_s(x) \le c_s(x).$$

In particular $h_s(x) = \infty$ implies $c_s(x) = \infty$. The first inequality uses $f \ge 0$; the second needs only $f \ge -10^4$.

Proof. If s is ineligible, then $c_s = \infty$ and there is nothing to show. Otherwise apply Lemma 11 with $t'' = t_\gamma(x) \le \hat{t}_s(x)$; this holds because tiers only take maxima. It gives $B_s \ge h_s(x)$; since $B_s < \infty$ for an eligible service, $h_s(x) = \infty$ forces $c_s(x) = \infty$. All amounts are nonnegative by A1. In particular, per-weight and percentage-of-base amounts are roundings of nonnegative numbers, so $\Sigma^{\mathsf{F}}, \Sigma^N \ge 0$ and $Q \ge 0$. By Lemma 12, $c_s^0 \ge \eta_f(B_s + \Sigma^{\mathsf{F}}) \ge \eta_f(h_s(x)) = \beta_s(x) \ge h_s(x)$. Finally $c_s = c_s^0 + [\![c_s^0 Q/100]\!] \ge c_s^0$, because the rounding of a nonnegative number is nonnegative. □

3.1 Surcharge-class refinement (RQ+)

Call a predicate *geometric for γ* if it reads only parcel-level features, or also *DAS* when Assumption A7 holds for γ. Let Π_γ be the set of geometric predicates that occur in the tiers, ineligibility predicates or surcharges of the services of γ. The class of x is $C(x) = \{\pi \in \Pi_\gamma : \pi(x)\}$.

Definition 14 (Class key)**.** For $s \in \gamma$, a local signature σ with $z = \sigma_s$, an interval e and a class C, let $\hat{e} = \max\big(\{e\} \cup \{e_\gamma(m_s^{(j)}) : \pi_s^{(j)} \in C\}\big)$. If $z = \perp$, if $\psi_s \in C$, or if $h_s(z, \hat{e}) = \infty$, set $\chi_s = \infty$. Otherwise

$$\chi_s(\sigma, e, C) = h_s(z, \hat{e}) + \sum_{(g,\varphi)} \max\big(\{0\} \cup \{A_r[z] : r \in \mathcal{R}_s \text{ fixed},\ \pi_r \in C,\ (g_r, \varphi_r) = (g, \varphi)\}\big).$$

Write $\chi_s = h_s(z, \hat{e}) + \chi_s^{\mathsf{F}} + \chi_s^N$, where χ_s^{F} and χ_s^N sum over the fuel and non-fuel keys. The row is *non-fuel-flagged* (NF) if some service with $\chi_s < \infty$ has $\chi_s^N > 0$. The bound is $\beta_s^+ = \chi_s$ for NF rows and $\beta_s^+ = \eta_f(\chi_s)$ otherwise.

Lemma 15 (L3 for RQ+)**.** *Assume A1–A3 and A7. For every x and $s \in \gamma$, $\beta_s^+(\sigma(x), e_x, C(x)) \le c_s(x)$.*

Proof. A geometric predicate has the same value in $C(x)$ and in the cost evaluation, because it reads only features that do not depend on the service; for *DAS* this uses A7. If $\chi_s = \infty$ because of $z = \perp$ or $\psi_s \in C(x)$, then s is ineligible. Let $t'' = \max\big(\{t_\gamma\} \cup \{m_s^{(j)} : \pi_s^{(j)} \in C(x)\}\big)$. Every tier in $C(x)$ fires in the evaluation, so $t'' \le \hat{t}_s(x)$. By Lemma 8(c), $e_\gamma(t'') = \hat{e}$. If s is eligible, Lemma 11 gives $B_s \ge h_s(z, \hat{e})$, and in particular $h_s(z, \hat{e}) < \infty$. If $h_s(z, \hat{e}) = \infty$, the same lemma shows that s is ineligible. For every key, the maximum over all firing rules of the key is at least the maximum over its firing fixed geometric rules, since all amounts are nonnegative. Hence $\Sigma^{\mathsf{F}} \ge \chi_s^{\mathsf{F}}$ and $\Sigma^N \ge \chi_s^N$. For an NF row, $c_s \ge c_s^0 \ge \eta_f(B_s + \Sigma^{\mathsf{F}}) + \Sigma^N \ge B_s + \Sigma^{\mathsf{F}} + \Sigma^N \ge \chi_s$. For a row that is not NF, every service with finite key has $\chi_s^N = 0$, so $\eta_f(\chi_s) = \eta_f(h + \chi_s^{\mathsf{F}}) \le \eta_f(B_s + \Sigma^{\mathsf{F}}) \le c_s^0 \le c_s$. □

4 Query Exactness and Instance Optimality (T4)

Definition 16 (Threshold algorithm TA)**.** On input x:

1. Locate the elementary key interval of κ and its signature σ. For every group γ, compute $t_\gamma(x)$, $e_x = e_\gamma(t_\gamma)$, the fuel ratio f_γ and the list $\mathcal{L}_\gamma = \mathcal{L}_\gamma(\sigma|_\gamma, e_x)$. Set a pointer p_γ to the first entry, and drop the group if its first entry has $h = \infty$.

2. Set $(best, arg) = (\infty, -1)$. While some group remains: let γ be a remaining group whose head has the smallest β. If $\beta_{\text{head}} > best$, stop. Otherwise evaluate $v = c_s(x)$ for the head s, and replace $(best, arg)$ by (v, s) if $v < best$, or if $v = best$ and $s < arg$. Advance p_γ, and drop γ if its list is exhausted or its next entry has $h = \infty$.

3. Return $(best, arg)$, where $(\infty, -1)$ encodes $\perp$.

Definition 17 (Access model)**.** An algorithm receives x and the index. A *sorted access* to a list returns the next entry (s, h_s), from which $\beta_s(x)$ follows, at cost c_S. An *evaluation* of s returns $c_s(x)$ at cost c_R. Nothing else about the costs is observable. The set of instances consistent with the bounds is $\mathcal{I}(\beta) = \{c' \in (\mathbb{N} \cup \{\infty\})^{\mathcal{S}} : c'_s \ge \beta_s(x)\ \forall s\}$. An algorithm is *correct* if every run returns $\min_{\text{lex}}$ of the finite pairs of c', or $\perp$, for every $c' \in \mathcal{I}(\beta)$. For randomized algorithms we require correctness of every run (Las Vegas). This is the sorted/random access model of threshold algorithms [1]. As in minimal probing [2], the grade of an entry is replaced by a lower bound and a probe reveals the exact value.

Theorem 18 (Exactness and instance optimality, T4)**.** *Assume A1–A4, and let $(c^*, s^*) = \mathrm{CFS}(x)$.*

(a) TA returns $\mathrm{CFS}(x)$. It agrees with the brute force that scans the services in identifier order and keeps a strictly smaller cost, including the tie rule and the answer $\perp$.

(b) If $\mathrm{CFS}(x) \ne \perp$, TA evaluates exactly the services in $\{s : \beta_s(x) \le c^*\}$. If $\mathrm{CFS}(x) = \perp$, it evaluates exactly those with $\beta_s(x) < \infty$. It performs at most (number of evaluations)$+|\Gamma|$ sorted accesses.

(c) Let $M(x) = \{s : (\beta_s, s) <_{\text{lex}} (c^*, s^*)\} \cup \{s^*\}$ if $\mathrm{CFS}(x) \ne \perp$, and $M(x) = \{s : \beta_s < \infty\}$ otherwise. Every run of every correct algorithm on any $c \in \mathcal{I}(\beta)$ with $\mathrm{CFS} = (c^*, s^*)$ evaluates every service of $M(x)$.

(d) Let $\tau(x) = |\{s > s^* : \beta_s(x) = c^*\}|$*, and let* $\tau(x) = 0$ *if* $\mathrm{CFS}(x) = \perp$*. TA makes exactly* $|M(x)| + \tau(x)$ *evaluations. For every correct algorithm* A*,*

$$\mathrm{cost}(\mathrm{TA}, x) \le \Big(1 + \frac{c_S}{c_R}\Big)\mathrm{cost}(A, x) + (c_R + c_S)\,\tau(x) + c_S|\Gamma|.$$

Hence, on the inputs with $\tau(x) = 0$*, TA is instance optimal in the sense of [1] with optimality ratio* $1 + c_S/c_R$ *and additive constant* $c_S|\Gamma|$*, and its number of evaluations is the minimum over all correct algorithms. If only evaluations are counted, the optimality ratio is* 1 *with additive term* $\tau(x)$*.*

Proof. Order of pops. Within a group, $\beta_s = \eta_f(h_s)$ with a common f (Lemma 6). By Lemma 12, every list is therefore nondecreasing in β. TA always pops the smallest head, so the sequence of popped bounds is nondecreasing, as in a merge of sorted lists.

Incumbent invariant. After every step, $(\mathit{best}, \mathit{arg})$ is the lexicographic minimum of the finite pairs of the evaluated services, or $(\infty, -1)$ if there is none. This holds initially. An infinite v never replaces the incumbent: if $\mathit{best} = \infty$, then $\mathit{arg} = -1$ and $s < -1$ fails. A finite v replaces it exactly when $(v, s) <_{\mathrm{lex}} (\mathit{best}, \mathit{arg})$.

(a) Consider the moment TA stops or runs out of groups. Let s be an unevaluated service. Either s lies after a dropped entry with $h = \infty$; then $h_s = \infty$ because the lists are sorted, and $c_s = \infty$ by Lemma 13. Or s lies in a remaining list; then $c_s \ge \beta_s \ge \beta_{\mathrm{head}} > \mathit{best}$. In both cases (c_s, s) is not lexicographically smaller than the incumbent, so by the invariant the incumbent is the lexicographic minimum of all finite pairs, which is $\mathrm{CFS}(x)$. The brute force computes the same minimum: it scans in identifier order by A4, and a strict comparison keeps the first, hence smallest, identifier among equal costs.

(b) Let $\mathrm{CFS}(x) = (c^*, s^*)$. When TA stops, $\mathit{best} = c^*$ by (a). Every service with finite h that was not evaluated lies in a remaining list and has $\beta_s > \mathit{best} = c^*$. If the loop ended because no group remained, every service with finite h was evaluated. Hence every s with $\beta_s \le c^*$ is evaluated. Conversely, let $\beta_s > c^*$. Since $\beta_{s^*} \le c_{s^*} = c^* < \beta_s$ and pops are nondecreasing, s^* is popped and evaluated before s. From then on $\mathit{best} \le c^*$, and best is the cost of an evaluated service, so $\mathit{best} \ge c^*$. When s becomes the smallest head, $\beta_s > \mathit{best}$ and TA stops. If $\mathrm{CFS}(x) = \perp$, then best stays ∞, the stop test never fires, and exactly the services with finite h, equivalently finite β, are evaluated. For the sorted accesses, each group reads its first entry once, and each evaluation is followed by at most one further read.

(c) Suppose a run of a correct A on c never evaluates some $s \in M(x)$. Define c' by $c'_u = c_u$ for $u \ne s$, and set $c'_s = \beta_s$ if $s \ne s^*$, or $c'_{s^*} = c^* + 1$ if $s = s^*$. If $s \ne s^*$, then $\beta_s \le c^* < \infty$, or $\beta_s < \infty$ in the case $\mathrm{CFS} = \perp$, so c'_s is finite. Thus $c' \in \mathcal{I}(\beta)$. Every sorted access and every evaluation of the run returns the same value under c' as under c: the bounds are unchanged, and s is never evaluated. Hence the run returns the same answer, (c^*, s^*) or $\perp$. But the correct answer for c' differs. If $s \ne s^*$ and $\mathrm{CFS}(x) \ne \perp$, then $(c'_s, s) = (\beta_s, s) <_{\mathrm{lex}} (c^*, s^*)$, so the minimum for c' is strictly smaller. If $\mathrm{CFS}(x) = \perp$, then c' has a finite pair. If $s = s^*$, every pair other than that of s^* is unchanged and differs from (c^*, s^*), and the pair of s^* is now $(c^* + 1, s^*)$; so no pair equals (c^*, s^*). This contradicts the correctness of A.

(d) We have $(\beta_s, s) <_{\mathrm{lex}} (c^*, s^*)$ iff $\beta_s < c^*$, or $\beta_s = c^*$ and $s < s^*$. Moreover, s^* belongs to the first set iff $\beta_{s^*} < c^*$. Hence $|M| = |\{\beta_s < c^*\}| + |\{\beta_s = c^*, s \le s^*\}|$, while by (b) TA evaluates $|\{\beta_s < c^*\}| + |\{\beta_s = c^*\}| = |M| + \tau$ services. Its cost is therefore at most $c_R(|M| + \tau) + c_S(|M| + \tau + |\Gamma|)$. By (c), $\mathrm{cost}(A, x) \ge c_R|M|$, which gives the inequality. □

Remark 19 (Tie term). The term $\tau(x)$ is unavoidable for any algorithm that pops services in bound order and stops on a strict comparison. Sorting the lists by (β, s) instead of (h, s) does not remove

it in general, because η_f may map distinct h to the same β, and a fuel-dependent order would contradict Lemma 6. On instances where no other service has a bound exactly equal to the optimal cost, $\tau = 0$ and TA is exactly optimal in evaluations.

Proposition 20 (RQ+ query)**.** *Assume A1–A4 and A7. TA run on the RQ+ rows, whose lists are sorted by (χ_s, s) with bound β^+, returns* CFS(x)*. Statements (b)–(d) of Theorem 18 hold with β^+ in place of β.*

Proof. Within a row, β^+ is either χ or $\eta_f(\chi)$ with a common f, so the rows are nondecreasing in β^+. Lemma 15 replaces Lemma 13, and a service with $\chi_s = \infty$ has $c_s = \infty$ by the proof of that lemma. The rest of the proof of Theorem 18 uses only these two properties. □

Proposition 21 (Batch query)**.** *Assume A1–A4. The vectorised algorithm, which in round r evaluates the rank-r entry of every group for every parcel that is still active in that group and deactivates a parcel in a group when the rank-r bound is ∞ or exceeds the current best, returns* CFS(x) *for every parcel.*

Proof. The incumbent invariant holds because the lexicographic-minimum update is commutative and associative. A group is deactivated for a parcel at rank r only if $\beta_r > best_{\text{now}} \geq best_{\text{final}}$ or $\beta_r = \infty$. All later ranks have bounds at least β_r, so their costs exceed the final best or are infinite. The argument of Theorem 18(a) then applies. □

Remark 22 (Truncated lists). With list truncation to the first κ entries, the implementation evaluates all remaining services of a group when the truncated list is exhausted. Exactness is preserved, because unlisted services have bounds at least as large as the last listed bound. Statement (b) of Theorem 18 no longer holds. The surcharge-class index requires untruncated lists (Appendix B, item M11).

5 Index Updates

Proposition 23 (Price edit)**.** *After $P_s[z, k] \leftarrow v$, the in-place repair recomputes $h_s(z, \cdot)$, and for every local signature with $\sigma_s = z$ and every e whose hull value changed, it removes s and reinserts it at the first position whose entry is lexicographically after (h^{new}, s). The resulting lists equal those of a fresh build. The repair takes $O(|\{\sigma \in \Sigma_\gamma : \sigma_s = z\}| \cdot |E^\Delta| \cdot n_\gamma + K_s + m_\gamma \log K_s)$ time with arrays, where E^Δ is the set of intervals whose hull value changed.*

Proof. The value $h_{s'}(z', e)$ depends only on (s', z', e) and on the prices of s'. Hence only the entries of s in lists with $\sigma_s = z$ can change, and only at intervals in E^Δ. Removal followed by insertion at the stated position restores the order by (h, s), which is strict because identifiers are distinct. The time bound counts the hull computation and one array shift per touched list. □

For RQ+, the edit invalidates the materialised rows of the local signatures with $\sigma_s = z$. These rows are rebuilt from the current data on their next access, so the answers remain exact by Proposition 20. A fuel edit changes only the schedule, which takes $O(1)$ index work beyond the array insertion, by Lemma 6.

Proposition 24 (Zone edit)**.** *After the edit $Z_s(\kappa) \leftarrow z'$ for $\kappa \in [l, u]$, the repair inserts the cuts l and $u + 1$ and, for every elementary interval inside $[l, u]$, replaces its signature by the signature with $\sigma_s = z'$. Existing global and local signatures are reused, and new local signatures receive freshly built lists. For every key, the resulting index returns the same lists as a fresh build of the edited instance, so Theorem 18 continues to hold.*

Proof. Outside $[l, u]$ no zone map changes. The new cuts make every zone map constant on every elementary interval (Proposition 5). A list depends only on the local zone vector and e, so reusing an existing local signature with the same vector, or building fresh lists from current data, yields the lists of a fresh build. A fresh build may number the signatures differently and merge adjacent intervals, which does not affect any answer. □

6 Exact Maintenance (T7)

Definition 25 (Top-2 state)**.** For a cost vector c of a parcel, let $F(c)$ be the set of finite pairs. Then $\mathrm{Top2}(c) = (p_1, p_2)$, where $p_1 = \min_{\mathrm{lex}} F(c)$ and $p_2 = \min_{\mathrm{lex}}(F(c) \setminus \{p_1\})$; missing entries equal the padding $\perp = (\infty, -1)$, which we order after every finite pair. We write $p_i = (c_i, s_i)$.

Definition 26 (Repair)**.** Given the state (p_1, p_2) of a parcel, a service s and its new cost c', let $p = (c', s)$:

(R1) $s = s_1$: if $c' < \infty$ and $p <_{\mathrm{lex}} p_2$, set $p_1 \leftarrow p$; otherwise *recompute*.

(R2) $s = s_2$: if $p <_{\mathrm{lex}} p_1$, set $(p_1, p_2) \leftarrow (p, p_1)$; else if $c' \le c_2$, set $p_2 \leftarrow p$; otherwise *recompute*.

(R3) $s \notin \{s_1, s_2\}$ and $c' < \infty$: if $p <_{\mathrm{lex}} p_1$, set $(p_1, p_2) \leftarrow (p, p_1)$; else if $p <_{\mathrm{lex}} p_2$, set $p_2 \leftarrow p$; otherwise do nothing.

(R4) $s \notin \{s_1, s_2\}$ and $c' = \infty$: do nothing.

To *recompute* means to set the state to Top2 of the current costs of all services.

Lemma 27 (Single-service repair)**.** *Let $(p_1, p_2) = \mathrm{Top2}(c)$, and let c' differ from c at most in coordinate s. Then the repair with c'_s yields* $\mathrm{Top2}(c')$.

Proof. We use the fact (F): every finite pair of c other than p_1, p_2 is lexicographically after p_2. Pairs of services other than s are the same under c and c'. (R1) The pair of s was p_1. If $c' < \infty$ and $p <_{\mathrm{lex}} p_2$, then p precedes p_2 and, by (F), every other pair, so $\mathrm{Top2}(c') = (p, p_2)$. The other cases are recomputed. (R2) The pair of s was $p_2 = (c_2, s)$. If $p <_{\mathrm{lex}} p_1$, then $p <_{\mathrm{lex}} p_1 <_{\mathrm{lex}}$ every other pair, so $\mathrm{Top2}(c') = (p, p_1)$. Otherwise $p_1 <_{\mathrm{lex}} p$, since $p \neq p_1$. If moreover $c' \le c_2$, then c' is finite and $p \le_{\mathrm{lex}} (c_2, s)$, which precedes every other pair by (F). Hence $\mathrm{Top2}(c') = (p_1, p)$. The remaining case is recomputed. (R3) The old pair of s, if finite, came after p_2. If $p <_{\mathrm{lex}} p_1$, then $\mathrm{Top2}(c') = (p, p_1)$. If $p_1 <_{\mathrm{lex}} p <_{\mathrm{lex}} p_2$, then it is (p_1, p). Otherwise p comes after p_2, and both the old and the new pair of s come after p_2, so the state is unchanged. (R4) The pair of s leaves F. It came after p_2, so the state is unchanged. □

Lemma 28 (Several edited services)**.** *Suppose that an edit changes the costs of $S_\Delta = \{u_1, \ldots, u_r\}$ from c to c', and let all repairs and recomputations use c'. If each parcel is repaired once for each $u \in S_\Delta$, in any order, its final state is* $\mathrm{Top2}(c')$.

Proof. We prove the invariant that after the repairs for $u_1, \ldots, u_i$, the state equals $\mathrm{Top2}(c^{(i)})$ for a vector $c^{(i)}$ with $c^{(i)}_u = c'_u$ for $u \notin S_\Delta$ and for $u \in \{u_1, \ldots, u_i\}$. For $i = 0$, take $c^{(0)} = c$, which agrees with c' outside S_Δ. For the step, if the repair for u_{i+1} does not recompute, Lemma 27 applied to $c^{(i)}$ and $c^{(i)}[u_{i+1} \leftarrow c'_{u_{i+1}}]$ yields the invariant. If it recomputes, the state becomes $\mathrm{Top2}(c')$, and $c^{(i+1)} = c'$ satisfies the invariant. For $i = r$ we have $c^{(r)} = c'$. □

Definition 29 (Supports)**.** Let X be the historical parcels, $t_\gamma(x)$ their pre-tier weights and $e_x = e_\gamma(t_\gamma(x))$. A *bucket* (σ, e) is the set of parcels whose signature is σ and whose interval in the group of the edited service is e.

(S1) *Cell* (s, z, k). Let $\bar{b} = b_{s,k}$, except $\bar{b} = +\infty$ if s extrapolates and $k = K_s$. Let $I_k = \{e \in \{1, \dots, m+1\} : b_{s,k-1} < E_e \le \bar{b}\}$. Then

$$U_{\text{cell}} = \{x : Z_s(\kappa) = z,\ e_x \in I_k\} \cup \bigcup_{j:\ b_{s,k-1} < m_s^{(j)} \le \bar{b}} \{x : Z_s(\kappa) = z,\ \pi_s^{(j)}(x)\}.$$

(S2) *Band* (s, k) *over zones* Z'*, or any set of cells of one service.* The union of the cell supports.

(S3) *Table rewrite of* s*, such as a general rate increase.* $U_{\text{srv}} = \{x : Z_s(\kappa) \neq \perp\}$.

(S4) *Rule edit of* s, which changes $\mathcal{R}_s$ or ψ_s but not zones, prices, the group or the tiers. If ψ_s changes, or if a changed rule is a percentage-of-total rule, the support is U_{srv}. Otherwise, let $\Delta\mathcal{R}$ be the symmetric multiset difference of the old and the new rule lists, comparing all fields of a rule. Then $U_{\text{rule}} = \{x \in U_{\text{srv}} : \exists r \in \Delta\mathcal{R},\ \pi_r(x)\}$, where π_r is evaluated as in Definition 2.

(S5) *Zone edit* $Z_s(\kappa) \leftarrow z'$ for $\kappa \in [l, u]$: $U_{\text{zone}} = \{x : \kappa \in [l, u],\ Z_s^{\text{old}}(\kappa) \neq z'\}$.

(S6) *Fuel edit* of class ϕ, segment w: let $D_w = [\sigma_{\phi,w}, \sigma_{\phi,w+1})$, with $\sigma_{\phi,0}$ replaced by $-\infty$ for $w = 0$ and $\sigma_{\phi,W} = +\infty$. Then $U_{\text{fuel}} = \{x : d(x) \in D_w\}$, and $S_\Delta = \{u : \phi_u = \phi\}$. Inserting a new start σ' into segment w gives the support $\{x : d(x) \in [\sigma', \sigma_{\phi,w+1})\}$.

(S7) *Service insertion or deletion:* U_{srv}, with the cost of the inserted service before the edit, or of the deleted service after it, taken as ∞.

Lemma 30 (Cell support)**.** *Assume A3, A5 and A6. If $c_s(x)$ changes when only $P_s[z, k]$ changes, then $x \in U_{\text{cell}}$.*

Proof. The cost reads the price table only through B_s, which is also the only input of percentage-of-base amounts. B_s reads $P_s[z, k]$ only if $Z_s(\kappa) = z$ and either (i) $k_s(\hat{t}) = k$ and s does not extrapolate at x, or (ii) s extrapolates with $\hat{t} > \lambda_s$ and $k = K_s$. Let $t = t_\gamma(x)$. By A5, firing of tiers does not depend on T, so $\hat{t} = \max(\{t\} \cup \{m_s^{(j)} : \pi_s^{(j)}(x)\})$.

Case $\hat{t} = t$. In case (i), if some breakpoint is at least $t - \epsilon$, Lemma 8(a) gives $b_{s,k-1} < E_{e_x} \le b_{s,k} \le \bar{b}$, so $e_x \in I_k$. Otherwise $k_s(t) = K_s = k$ and $t - \epsilon > b_{s,K_s}$ by Lemma 8(d). Then a non-extrapolating s is ineligible both before and after the edit, so its cost cannot change. If s extrapolates, then $\bar{b} = \infty$ and $E_{e_x} \ge t - \epsilon > b_{s,K_s} - \epsilon \ge b_{s,K_s-1}$; for $e_x = m + 1$ this holds trivially, and otherwise it uses A6. So $e_x \in I_{K_s}$. In case (ii), $t > \lambda_s \ge b_{s,K_s}$ by A3, and the same argument gives $e_x \in I_{K_s}$.

Case $\hat{t} > t$. Then $\hat{t} = m_s^{(j)}$ for a firing tier j. In case (i), $k_s(m_s^{(j)}) = k$ means $b_{s,k-1} < m_s^{(j)} - \epsilon$ and, if $k < K_s$ or no clipping occurs, $m_s^{(j)} - \epsilon \le b_{s,k}$. Hence $b_{s,k-1} < m_s^{(j)}$, and $m_s^{(j)} \le b_{s,k} + \epsilon$, which together with A5 gives $m_s^{(j)} \le b_{s,k}$. If clipping occurs, then $k = K_s$, and either s extrapolates and $\bar{b} = \infty$, or s is ineligible both before and after the edit. In case (ii), $m_s^{(j)} > \lambda_s \ge b_{s,K_s} > b_{s,K_s-1}$ and $\bar{b} = \infty$. In every case $b_{s,k-1} < m_s^{(j)} \le \bar{b}$, and x belongs to the tier term of U_{cell}, because by A5 the tier predicate evaluated without service-dependent features agrees with the evaluation. □

Lemma 31 (Other supports)**.** *If the costs change on parcel x under an edit of type (S2)–(S7), then x lies in the corresponding support.*

Proof. (S2) A set of cells of one service changes $c_s(x)$ only if one of the cells is read at x; apply Lemma 30 to each cell. (S3) If $Z_s(\kappa) = \perp$, then $c_s(x) = \infty$ before and after the edit. (S4) The zones, $\hat{t}$ and B_s are unchanged. If x is in no support, then no rule of $\Delta\mathcal{R}$ fires at x. The firing rules at x, with their amounts and keys, are then the same before and after the edit, because the unchanged rules are identical and the changed ones do not fire. Since every $\mu_{g,\varphi}$ is a maximum over firing rules and ψ_s is unchanged, (1) is unchanged. (S5) c_s reads κ only through $Z_s(\kappa)$, which also supplies the *ZONE* feature, and through $DAS_s(\kappa)$, which is unchanged. (S6) $f_\phi(d)$ changes exactly for $d \in D_w$, and the edit changes no other input of any service. (S7) A service with $Z_s = \perp$ has cost ∞ before and after. □

Theorem 32 (Exact maintenance, T7)**.** *Let an edit Δ change the services in S_Δ, and let $D_\Delta = \{x \in X : \exists u \in S_\Delta,\ c_u(x) \neq c'_u(x)\}$. For every U with $D_\Delta \subseteq U \subseteq X$, repairing every $x \in U$ once for each $u \in S_\Delta$, in any order and with the edited instance, makes the state of every $x \in X$ equal to* $\mathrm{Top2}(c'(x))$*. Under A3, A5 and A6, the supports of Definition 29 contain D_Δ. The repair evaluates $|S_\Delta| \cdot |U|$ costs and recomputes the set $R \subseteq U$ of parcels that reach a recompute branch, at a cost of $|\mathcal{S}|$ evaluations each. A parcel reaches a recompute branch only if the edited service was its best and became infeasible or lexicographically worse than the runner-up (R1), or was its runner-up and became more expensive than before without overtaking the best (R2).*

Proof. For $x \notin U$, all costs are unchanged, so the old state equals $\mathrm{Top2}(c'(x))$. For $x \in U$, apply Lemma 28. The containment of the supports is Lemmas 30 and 31. The cost statement counts one evaluation per parcel and service, plus one full recomputation per element of R. The characterisation of R follows from (R1) and (R2). □

Remark 33 (Granularity). U_{cell} is a union of whole buckets plus tier sets. A parcel in a selected bucket whose tier moves it out of band k, or for which s is ineligible, is included although its cost does not change; its repair is a no-op. Exactness never depends on minimality.

7 Complexity

Let $n = |\mathcal{S}|$, $N \leq 1 + \sum_s R_s$ the number of elementary key intervals, Γ the set of groups, n_γ the size of group γ, $m_\gamma = |E_\gamma| \leq \sum_{s \in \gamma} K_s$, Σ_γ the local signatures, and Z_s^γ the number of distinct zones of s that occur in Σ_γ. Let $C_{\mathrm{ev}}(s)$ be the cost of one evaluation, which is $O(\log R_s + \log K_s + \log |DAS_s| + \sum_r |\pi_r| + |\pi_s^{(\cdot)}| + |\psi_s|)$, where $|\pi|$ is the number of conditions of a predicate.

Proposition 34 (Build, space, query and maintenance cost)**.** *(a)* Build: *the index is built in time*

$$O\Big(nN \log N + \sum_\gamma \Big[\sum_{s \in \gamma} Z_s^\gamma (K_s + m_\gamma \log K_s) + |\Sigma_\gamma|(m_\gamma + 1)\, n_\gamma \log n_\gamma\Big]\Big).$$

(b) Space: $O\big(N + n|\Sigma| + \sum_\gamma |\Sigma_\gamma|(m_\gamma + 1)\, n_\gamma\big)$ *words. RQ+ adds $O(n_\gamma)$ words per materialised row (σ, e, C), and the number of such rows never exceeds the number of distinct triples $(\sigma|_\gamma, e_x, C(x))$ among the queried parcels.*

(c) Query (TA): $O\big(\log N + \sum_\gamma (C_\tau + \log m_\gamma) + N_{\mathrm{TA}}(x)\,(|\Gamma| + C_{\mathrm{ev}})\big)$*, where C_τ is the cost of computing t_γ and $N_{\mathrm{TA}}(x) = |\{s : \beta_s(x) \leq c^*\}|$ by Theorem 18(b). With a heap over the group heads, the factor $|\Gamma|$ becomes $\log |\Gamma|$.*

(d) Maintenance: *initialisation takes $O(|X| \sum_s C_{\mathrm{ev}}(s) + |\Gamma|\,|X| \log |X|)$. Enumerating U_{cell} takes $O(|\{\sigma : \sigma_s = z\}| \cdot |I_k| + |U| \log |U|)$ given the cached tier sets. The repair costs $|U|$ evaluations of s plus $|R| \cdot n$ evaluations.*

Proof. (a) Computing the zone vectors takes n binary searches per elementary interval, and deduplication is a sort of N vectors of length n. For every service and every distinct zone, the suffix minimum takes $O(K_s)$, and locating the m_γ intervals takes $O(m_\gamma \log K_s)$. Sorting takes $O(n_\gamma \log n_\gamma)$ per list. (b) Count the stored arrays. (c) Point location and the per-group set-up are followed, for each evaluated service, by one selection among at most $|\Gamma|$ heads and one evaluation; the number of pops is N_{TA}. (d) The initial top-2 evaluates every service on every parcel, and the buckets are sorted once per group. Enumeration performs one dictionary lookup per candidate bucket and deduplicates the result. □

8 Envelope Complexity and General Predicates (T5, T6)

Throughout this section, fix a group γ, a local signature, the fuel ratio and the values of all predicates. Assume that all surcharges are fixed amounts and that no service extrapolates. Then $c_s(x)$ depends on the weight only through $k_s(\hat{t}_s)$. Define the *restricted fragment* by the additional condition that no tier fires, so that $\hat{t}_s = t_\gamma$.

Proposition 35 (One-dimensional envelope, T5 part 1)**.** *In the restricted fragment, the map $t \mapsto$* CFS *restricted to γ is constant on each elementary interval $\{t : e_\gamma(t) = e\}$, so it has at most $m_\gamma + 1 \le \sum_{s\in\gamma} K_s + 1$ pieces. The bound $m_\gamma + 1$ is attained for every m_γ.*

Proof. By Lemma 8(a), $k_s(t)$ is determined by $e_\gamma(t)$ for every $s \in \gamma$, so all costs are constant on each elementary interval. For tightness, take services $s_1, \dots, s_m$ with s_1 having the single breakpoint 1 and price 1, and s_i for $i \ge 2$ having breakpoints $i-1, i$ and prices $3, 1$. On $(i-1, i]$ the service s_i costs 1, every $s_{i'}$ with $i' > i$ costs 3, and every $s_{i'}$ with $i' < i$ is ineligible. Hence the argmin is s_i on $(i-1, i]$, and $\perp$ on (m, ∞). This gives $m+1$ pieces with $E_\gamma = \{1, \dots, m\}$. □

Proposition 36 (Two-dimensional overlay, T5 part 2)**.** *For every rounding rule, $t_\gamma(a, V) = \max(\alpha_\gamma(a), \nu_\gamma(V))$ with nondecreasing $\alpha_\gamma, \nu_\gamma$. Hence, across all groups in the restricted fragment, the plane (a, V) is partitioned into at most $O(\Lambda^2)$ cells on which every cost is constant, where $\Lambda = \sum_\gamma (m_\gamma + 1)$. There are instances with two groups where the overlay has $\Omega(\Lambda^2)$ cells.*

Proof. For the carrier rule, $\alpha(a) = a$ and $\nu(V) = [V > \theta]V/D$. For the erp-style rule, the rounding of a and of V are applied separately and then maximised. For the enterprise rule in mode 1, $r(\max(a, v)) = \max(r(a), r(v))$ because r is nondecreasing; modes 2 and 3 are the special cases with a constant ν or α. Hence $\{t_\gamma \le w\} = \{\alpha(a) \le w\} \cap \{\nu(V) \le w\}$ is a down-closed axis-parallel rectangle, and the boundary between consecutive elementary intervals consists of at most one vertical and one horizontal segment or ray. The overlay of all groups is therefore an arrangement of at most 2Λ axis-parallel segments. Such an arrangement has $O(\Lambda^2)$ vertices and hence, by Euler's formula, $O(\Lambda^2)$ faces [5]. Every cost is constant on each face.

For the lower bound, take two carrier groups with $\theta = 0$, divisors $D_1 > D_2$ with $D_1/D_2 > 2K$, and $E_1 = E_2 = \{1, \dots, K\}$. In the wedge $V/D_1 < a < V/D_2$ we have $t_1 = a$ and $t_2 = V/D_2$. For $1 \le i < j \le K$, the point $(i - \frac{1}{2}, (j - \frac{1}{2})D_2)$ lies in the wedge, because $(j - \frac{1}{2})D_2/D_1 < 1/2 \le i - \frac{1}{2} < j - \frac{1}{2}$. Its pair of intervals is (i, j). Hence at least $K(K-1)/2 = \Omega(\Lambda^2)$ distinct cells occur. □

Conjecture 37 (T5 lower bound for the argmin)**.** *There are instances in the restricted fragment where the number of maximal connected regions of the (a, V) plane with a constant argmin is $\Omega(\Lambda^2)$.*

Proposition 36 shows this lower bound for the overlay of cells, not for the argmin. With one service per group, the argmin regions are separated by a threshold on the two price sequences, and a checkerboard pattern of the argmin is impossible. A construction with $\Theta(K)$ services per group is a plausible route.

Proposition 38 (Class counts, T6 part 1)**.** *(a) If the predicates of γ are conjunctions and disjunctions of threshold conditions on the parcel-level features, with h_ℓ distinct thresholds on feature ℓ and g_c distinct values that matter for categorical feature c, then $|\{C(x)\}| \le \prod_\ell (h_\ell + 1) \prod_c g_c$. (b) If the predicates are H halfspaces in $\mathbb{R}^d$ in general position, then the number of classes is at least $\sum_{i=0}^{d} \binom{H}{i} = \Theta(H^d)$ for fixed d. Every index that materialises one row per class eagerly therefore needs $\Omega(H^d)$ rows.*

Proof. (a) A class is determined by the cell of the grid of thresholds and by the categorical values, because every condition is constant on each grid cell. (b) Distinct full-dimensional cells of the arrangement have distinct sign vectors, and their number is $\sum_{i \le d} \binom{H}{i}$ [3, 4]. □

Conjecture 39 (T6)**.** *For general linear predicates in d dimensions, there is no exact CFS index of size polynomial in H and n, independent of d, that answers queries with $o(n)$ evaluations in the worst case.*

Proposition 38(b) excludes only eager per-class materialisation, which is the design that RQ+ avoids by building rows lazily. A general lower bound would require a cell-probe argument, which we do not claim.

A Correspondence with the Code

Line numbers refer to the code as of 2026-11-09, in `code/rateq/` unless stated otherwise.

Theory	Implementation
Zone map Z_s (Def. 1(i))	`model.py:151-154` `zone_of`; scalar `index.py:78-83`
Billable weight τ_γ	`model.py:216-231` `billable`; `index.py:58-73` `billable1`
Group key $\gamma(s)$	`model.py:147-149` `group_key`
Tiers, lower first	`model.py:142-145` `min_tiers`, `model.py:243-245`, `index.py:92-95`
Band k_s with $\epsilon = 10^{-9}$	`model.py:255-256`, `index.py:103-105`
Extrapolation with cap	`model.py:249-263`, `index.py:99-112`
Eligibility	`model.py:246-248, 254, 264`, `index.py:78-114`
Surcharges, keys, maxima	`model.py:269-288`; `index.py:115-137`
Cost (1)	`model.py:289-293`; `index.py:138-144`
CFS, brute force	`model.py:299-309`, `index.py:159-165`
Key partition, signatures	`index.py:182-193`
E_γ, hull h, lists	`index.py:200, 207-222, 224-244`
TA (Def. 16)	`index.py:258-299`; stop test `index.py:277`
Batch (Prop. 21)	`index.py:301-358`
Price and zone edits	`index.py:361-393, 407-458`
Class key χ, NF flag	`index.py:552-586`; geometric features `index.py:461-470`
RQ+ TA	`index.py:602-637`, `index.py:639-696`
Top-2, repair (R1)–(R4)	`maintain.py:21-32, 94-126`
U_{cell}, U_{srv}, U_{fuel}	`maintain.py:64-84, 86-88, 90-92`
U_{rule}	`code/erp_pilot.py:160, 171-185, 196-204`

Branch equivalence of the repair. The code encodes $\perp$ as $(\infty, -1)$, and its comparison treats $(\infty, -1)$ as smaller than (∞, s) for $s \geq 0$. Every branch nevertheless coincides with Definition 26. For $s = s_1$, the code keeps p iff $\neg((c_2, s_2) <_{\text{lex}} (c', s))$. If $c' = \infty$, this is false whether or not p_2 is the padding, so the code recomputes, as in (R1). For $s = s_2$, the promotion test $(c', s) <_{\text{lex}} (c_1, s_1)$ is false for $c' = \infty$, because c_1 is finite, and $c' \leq c_2$ is false as well, so the code recomputes. For other services with $c' = \infty$, both tests are false, as in (R4).

Assumption check. A scan of the compiled instances (the public cards, their scaled variants, the synthetic and erp-style generators, and the enterprise US instance) found no violation of A1–A7. In detail: no negative price, amount, coefficient or fuel ratio; no extrapolating service with $\lambda_s < b_{s,K_s}$; the identifier of every service equals its position; no tier predicate reads T, *ZONE*, *ZIP* or *DAS*; no tier minimum lies within ϵ above a breakpoint; and every group that shares a delivery-area start array also shares the level array.

B Code–Theory Discrepancies

None of the following items invalidates a reported result. Each is either a latent precondition that holds on all instances, or a superset that affects only efficiency.

M1 **Stop rule and ties** (`index.py:277, 622`). The strict test $\beta_{\text{head}} > \mathit{best}$ evaluates all services with bound equal to c^*. This is correct and gives the additive term $\tau(x)$ of Theorem 18(d). The research plan claims that every certificate-producing algorithm must evaluate $|\{s : \beta_s \leq c^*\}|$ services; the correct lower bound is $|M(x)|$, which is smaller by $\tau(x)$.

M2 **One extra interval in** U_{cell} (`maintain.py:69`). The condition $E_{e-1} < hi + 10^{-12}$ admits the interval directly after $b_{s,k}$, so the code enumerates $I_k \cup \{e_{hi} + 1\}$. This is a sound superset that slightly enlarges U.

M3 **Tier band test** (`maintain.py:79`) uses $lo < m \leq hi$ without ϵ. It is equivalent to the ϵ-band membership under the second part of A5.

M4 **Tier masks** (`maintain.py:56-62`) evaluate the tier predicate with the pre-tier T, with $\mathit{ZONE} = 0$, and with a cache keyed by the identity of the predicate object, which is shared across services. They are exact only under the first part of A5. A tier predicate that reads T, such as an oversize test that depends on the raised weight, would require evaluating the masks per service in tier order.

M5 **Stale signatures after zone edits.** The locations are `maintain.py:40-52` and `index.py:407-450`. The maintainer class computes the parcel signatures and the buckets only once. After `RQIndex.update_zone`, `support_cell` would read the old signatures and could miss parcels whose new zone equals z. There is also no Q3 routine for zone edits; the theory supplies U_{zone} (S5). The pilots never interleave zone edits with Q3, so their results are unaffected. A fix is to rebuild `sg` and the buckets of the affected keys after a zone edit.

M6 **Fuel support at the ends of the schedule.** Location: `code/pilot.py:239-240` and `maintain.py:90-92`. The support $[\sigma_w, \sigma_{w+1} - 1]$ is exact for $1 \leq w \leq W - 2$. For $w = 0$ it misses days before σ_0, which also use $f_{\phi,0}$ (`model.py:172`). For $w = W - 1$ it raises an index error. The pilot selects $w = 0$ only for schedules with at most two segments, and all fuel classes of the instances have more, so its results are unaffected.

M7 **Rule-level support** (`code/erp_pilot.py:160, 171-185`). The features use only the upper tier `oversize`, not `oversize2` and not the tier order, and the rule identity `rkey` omits the fuel flag. Both are sound on the enterprise data, which has no tiers and only rules without fuel, but they violate (S4) in general.

M8 **Negative amounts** (`index.py:131` vs. `model.py:283`). The scalar evaluator initialises group maxima with -1, so a group with only negative amounts contributes 0, whereas the vector evaluator contributes the negative maximum. The two coincide under A1.

M9 **Extrapolation limit** (`index.py:217-221`). The hull assumes A3. If $\lambda_s < b_{s,K_s}$, a parcel with $\hat{t} \in (\lambda_s, b_{s,K_s}]$ has $B_s = \lceil P_s[z, K_s]\hat{t}/b_{s,K_s} \rceil$, which may be below h, and Lemma 13 would fail. A guard that rejects such services at build time is advisable.

M10 **Shared delivery areas** (`index.py:486-487`). The flag `das_ok` compares only the identity of the start arrays, not of the level arrays (A7).

M11 **RQ+ with truncation** (`index.py:502-503`). The unsorted hull is recovered from the possibly truncated lists, so RQ+ requires `kappa=None`, which is the default.

M12 **Documentation** (`model.py:9-12`, `index.py:474-476`). The docstrings claim upward-closed predicates and monotone cost, and list only L, M, S, G, V, A as geometric features, whereas `GEOM` also contains *res*, $L + M + S$, day and month. The proofs use neither monotonicity nor the shorter list, so only the documentation is outdated.

References

[1] R. Fagin, A. Lotem, and M. Naor, "Optimal aggregation algorithms for middleware," *J. Comput. Syst. Sci.*, vol. 66, no. 4, pp. 614–656, 2003.

[2] K. C.-C. Chang and S.-w. Hwang, "Minimal probing: Supporting expensive predicates for top-k queries," in *Proc. ACM SIGMOD*, 2002, pp. 346–357.

[3] R. C. Buck, "Partition of space," *Amer. Math. Monthly*, vol. 50, no. 9, pp. 541–544, 1943.

[4] T. Zaslavsky, "Facing up to arrangements: Face-count formulas for partitions of space by hyperplanes," *Mem. Amer. Math. Soc.*, vol. 1, no. 154, 1975.

[5] M. de Berg, O. Cheong, M. van Kreveld, and M. Overmars, *Computational Geometry: Algorithms and Applications*, 3rd ed. Springer, 2008.